

Research & Documentation on SQL Server Views

1. Standard View

What is it?

A **Standard View** is a virtual table in SQL Server created using a SELECT query on one or more base tables. It does not store data physically but provides a way to simplify complex queries or restrict access to specific columns and rows.

Key Differences:

- Does **not** store data physically.
- Does **not** support indexes (except via indexed view technique).
- Mainly used for **query simplification** and **security**.

Real-life Use Cases:

- **Banking:** A view showing customers' basic information without sensitive details like account numbers.
- **University System:** A view showing enrolled students per course without revealing their contact details.
- **E-commerce:** A product view showing only in-stock items and their prices.

Limitations & Performance Considerations:

- Slower for large datasets as the data is retrieved dynamically.
 - Cannot always be updated directly (e.g., views with GROUP BY, JOINS, DISTINCT).
 - Does not improve performance unless used to simplify complex logic.
-

2. Indexed View

What is it?

An **Indexed View** (also called a materialized view) stores the result of the view **physically on disk** and maintains a **unique clustered index**. It improves query performance for complex joins and aggregations.

Key Differences:

- **Physically stores** the data.

- Supports indexing, which improves performance.
- Has stricter **creation and maintenance rules** compared to standard views.

Real-life Use Cases:

- **Banking:** Summary of total balance per customer across all accounts, updated in real-time.
- **E-commerce:** Daily sales summary per product category.
- **University System:** Average grades per course and semester.

Limitations & Performance Considerations:

- Can impact performance due to overhead of maintaining the index on data change.
 - Must follow **schema binding**, deterministic functions, and other strict requirements.
 - Not all queries are eligible for indexed views (e.g., use of outer joins, subqueries).
-

3. Partitioned View (Union View)

What is it?

A **Partitioned View** is a union of multiple tables (often with identical schema), typically used to simulate table partitioning across databases or servers. It logically divides data based on a partitioning column.

Key Differences:

- Combines data **horizontally** from multiple tables using UNION ALL.
- Used to manage large datasets across partitions or servers.
- Can support **distributed partitioned views** (across servers) with certain conditions.

Real-life Use Cases:

- **Banking:** Transaction history split by year into separate tables (e.g., Transactions_2023, Transactions_2024) and combined in a partitioned view.
- **E-commerce:** Sales data split by region or store.
- **University System:** Student records partitioned by batch year.

Limitations & Performance Considerations:

- Manual maintenance required; not automatic like table partitioning.
- **Check constraints** must be defined for optimization (partition elimination).
- Updates require extra care to ensure they target the correct base table.

View Type	Stores Data	Indexing Supported	Use Case Example	Limitations
Standard View	No	No	Hide sensitive customer data	Limited update support, slower for large queries
Indexed View	Yes	Yes (Clustered)	Real-time balance summaries	Complex to maintain, strict rules
Partitioned View	No (Union)	No (but can optimize)	Year-wise transaction data	Manual partitions, requires check constraints

1. The types of views:

View Type	DML Allowed?	Notes
Standard View	Yes	Allowed if the view is simple and directly maps to base tables.
Indexed View	Yes	Allowed under strict conditions and when the base tables allow it.
Partitioned View	Yes (<i>with restrictions</i>)	For local partitioned views , DML is allowed. For distributed partitioned views , DML requires special setup and constraints.

2. Restrictions and Limitations When Performing DML on Views

Allowed When the View:

- Refers to a **single base table**.
- Does **not use** aggregate functions (SUM, AVG), DISTINCT, GROUP BY, or UNION.
- Does **not include** joins or subqueries.
- Does **not include** set operations like UNION, INTERSECT, or EXCEPT.

Not Allowed or Restricted When:

- The view contains **JOINS** (especially outer joins).
- It uses **computed columns**, **aggregates**, or **DISTINCT**.
- It involves **read-only functions**, TOP, OFFSET-FETCH, or ORDER BY.
- The view is defined with **WITH CHECK OPTION** and the update violates the condition.
- For **partitioned views**, each table must have **CHECK constraints** and consistent schema.

3. Real-Life Example: Updating a View

Example-HR systems

```
CREATE VIEW vw_EmployeeDetails
AS
SELECT EmployeeID, Name, DepartmentID
FROM Employees
WHERE Status = 'Active';
```

- Simplifying JOIN-Heavy Queries with Views

In large database systems like banking, queries often involve multiple JOINS across related tables (e.g., Customers → Accounts → Transactions). These JOINS can become long, repetitive, and hard to maintain—especially when the same logic is reused in reports, dashboards, or by support teams.

By creating a View, you can:

- Encapsulate complex logic once.
- Reuse the view like a table in future queries.
- Improve readability, maintenance, and security by controlling what's exposed.

Create View:

```
CREATE VIEW vw_CustomerAccounts AS
SELECT
    c.CustomerID,
    c.FullName,
    c.Email,
    a.AccountID,
    a.AccountType,
    a.Balance
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID;
```

```

SELECT
    FullName,
    Email,
    AccountID,
    AccountType,
    Balance
FROM vw_CustomerAccounts
WHERE AccountType = 'Savings';

```

Without View:

```

SELECT
    c.FullName,
    c.Email,
    a.AccountID,
    a.AccountType,
    a.Balance
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID
WHERE a.AccountType = 'Savings';

```

Summary of View:

Benefit	Explanation
Cleaner SQL Code	No need to repeat JOIN logic
Easy Maintenance	Update logic in one view, not everywhere
Better Collaboration	Easier for support teams and analysts to use safely
Reusability	Can be used in reports, dashboards, procedures, etc.